

Plan de Proyecto

EcoSort: Clasificador de Residuos Inteligente

Universidad Nacional de La Plata



**FACULTAD
DE INGENIERÍA**



**FACULTAD DE
INFORMÁTICA**

Autores

Alvarez, Juan David - 03100/2

Estrada, Francisco - 02999/3

Taini, Micaela - 03345/2



1.Introducción

La gestión de residuos sólidos urbanos depende hoy, en la mayoría de los casos, de que la persona que descarta un residuo lo separe correctamente por su cuenta, esto es algo que en la práctica falla con frecuencia por desconocimiento, apuro o simple comodidad, y termina degradando la calidad de todo el proceso de reciclaje aguas abajo.

EcoSort aborda este problema con un punto de reciclaje inteligente: un dispositivo autocontenido que identifica mediante visión artificial (cámara) la categoría de un residuo (plástico, papel, vidrio u orgánico) en el momento en que se deposita, y lo deriva automáticamente al contenedor físico correcto, sin intervención humana en la clasificación. Toda la inferencia se procesa localmente en una Raspberry Pi 3, sin depender de conectividad a internet ni de servicios externos, funcionando de forma autónoma.

2.Objetivo

Objetivo General: Construir un punto de reciclaje inteligente que, a partir de una imagen capturada al momento de descartar un residuo, lo clasifique en una de 4 categorías (plástico, papel, vidrio, orgánico) y lo derive físicamente al contenedor correspondiente de forma automática, corriendo la inferencia íntegramente en el edge (Raspberry Pi 3, sin internet).

Objetivos Específicos:

- Entrenamiento del modelo de visión artificial: Recopilar y procesar un conjunto de datos (dataset) de imágenes de residuos para entrenar un modelo de detección y clasificación de objetos (como YOLO) capaz de discriminar entre las 4 categorías establecidas con un alto grado de precisión.
- Optimización para Edge Computing: Adaptar y convertir el modelo de inferencia entrenado a un formato ligero como TensorFlow Lite para asegurar que se ejecute eficientemente dentro de las limitaciones de procesamiento y memoria RAM de una Raspberry Pi 3, garantizando una latencia baja en la operación *offline*.
- Investigación de la estrategia de procesamiento del modelo (edge vs. host externo): Evaluar si conviene ejecutar la inferencia íntegramente en la Raspberry Pi 3, o derivar parte del procesamiento al dispositivo externo de la



red local, considerando la latencia extremo a extremo, la capacidad de cómputo disponible y la cantidad de residuos que el sistema debe poder clasificar por unidad de tiempo.

- Desarrollo del software de Control: Programar los scripts de control que coordinen el flujo del sistema de manera local: captura de la imagen con la cámara, ejecución de la inferencia del modelo y el envío de las señales de control (PWM) a través de los pines GPIO hacia los actuadores mecánicos.
- Implementación de interfaz de monitoreo (Dashboard local): Desarrollar un servidor web ligero embebido (utilizando herramientas como Flask) accesible vía red local, que permita visualizar el estado operativo del clasificador, consultar estadísticas de los residuos procesados a través de los datos enviados por el nodo emisor (Raspberry Pi).
- Diseño e implementación de la estructura electromecánica: Diseñar y construir el prototipo físico del punto de reciclaje, incorporando los servomotores y mecanismos necesarios para dirigir físicamente los residuos a los 4 contenedores (plástico, papel, vidrio, orgánico) según la señal recibida.

3.REQUERIMIENTOS FUNCIONALES

3.1 Captura de imágenes: El sistema debe ser capaz de capturar imágenes en tiempo real de los residuos ingresados utilizando la cámara USB o el módulo de cámara de la Raspberry Pi.

3.2 Clasificación de residuos: El sistema debe procesar la imagen capturada e identificar el tipo de residuo utilizando un modelo de visión artificial (YOLO/TFLite).

3.3 Accionamiento mecánico: Una vez clasificado el residuo, el sistema debe enviar una señal al servomotor para que mueva la estructura física y dirija el desecho al contenedor correspondiente.

3.4 Interfaz de monitoreo local (Dashboard): El sistema debe levantar un servidor web local (utilizando Flask) que permita monitorear el estado del clasificador, ver estadísticas de los residuos o el log de actividad a través de la red local.

3.5 Disponibilidad sin conectividad externa: el sistema debe funcionar de forma completamente autónoma, sin depender de una conexión a internet — tanto la inferencia como el broker MQTT y el dashboard corren de forma local en la red que arma la propia Raspberry Pi.



3.6 Estabilidad eléctrica: el sistema no debe reiniciarse ni degradar su funcionamiento por picos de corriente de los servos, lo cual exige mantener las dos ramas de alimentación (Pi y servos) eléctricamente separadas con GND común.

4.REQUERIMIENTOS NO FUNCIONALES:

4.1 Rendimiento / Latencia: el tiempo entre la captura de la imagen y la señalización del contenedor no debe superar un umbral que establezcamos como por ejemplo de 2-4 segundos, para que la espera no sea perceptible por el usuario.

4.2 Robustez ante condiciones de iluminación variables: el modelo debe mantener un nivel de precisión aceptable ante variaciones de iluminación y posición del objeto frente a la cámara, dado que no va a operar en un entorno controlado como un laboratorio.

4.3 Seguridad de acceso a la red local: al operar la Pi como access point (192.168.20.0/28), la red debe estar protegida con contraseña, dado que cualquier persona dentro del rango de wifi podría intentar acceder al dashboard o al broker MQTT si la red queda abierta.

4.4 Robustez Física: El diseño del punto de reciclaje y el torque del servomotor deben ser adecuados para soportar el peso y la caída de los residuos típicos (plásticos, vidrios, orgánicos) sin trabarse o descalibrarse.



5. Identificación de Partes

Listado de materiales a utilizar:

ITEM	OBJETIVO	CANTIDAD
Raspberry PI 3	Procesador, inferencia y servidor	1
Fuente oficial Pi 3 (5V/2.5A, micro-USB)	Alimentación para la PI	1
MicroSD 16–32GB clase A1/A2	Unidad de almacenamiento/ FileSystem.	1
Cámara USB	Captura de Imagen, componente clave en el flujo de trabajo.	1
Servo SG90	Acciona la compuerta de cada clase de elemento (plástico / papel / vidrio)	4
Fuente externa 5V $\geq 2-3$ A regulada	Alimenta los 4 servos, separado de la Pi	1
Plataforma de modelado	Cartón corrugado doble capa o impresión 3D — mecanismo de clasificación (depende de la etapa de desarrollo en la que nos encontremos)	

6. Componentes de Hardware a utilizar

Servo SG90



MicroSD 16–32GB clase A1/A2



Cámara (USB o módulo Pi Camera)



Raspberry PI 3



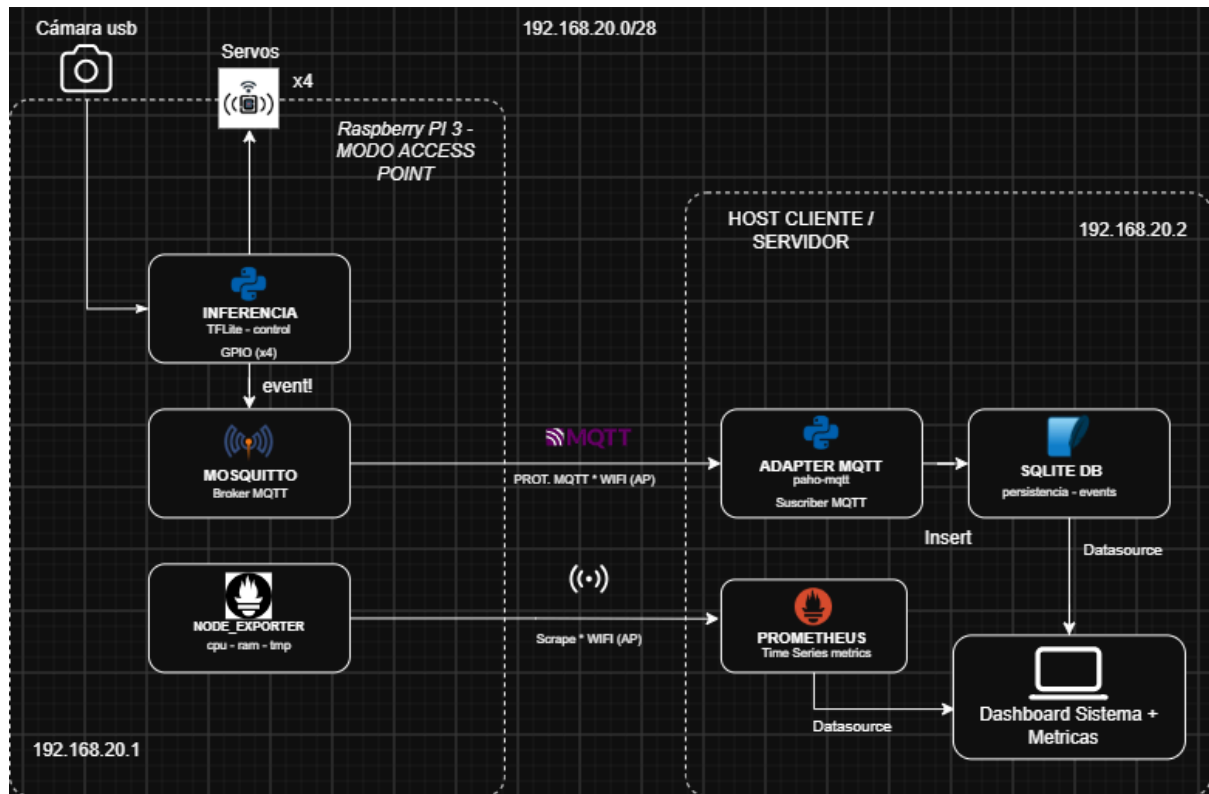
Fuente oficial Pi 3 (5V/2.5A, micro-USB)



Fuente externa 5V $\geq 2-3$ A regulada



7. Esquema Gráfico del Proyecto



[Esquema del Circuito de Alimentación](#)

8. Software a Desarrollar

El software del sistema se divide en dos capas según dónde corre: la Raspberry Pi 3 (captura, inferencia y actuación mecánica) y un dispositivo externo dentro de la misma red local (a cargo de la ruta fría — persistencia y visualización). Ambas capas se comunican por MQTT, sin que el tráfico salga en ningún momento de la red que arma la propia Pi.

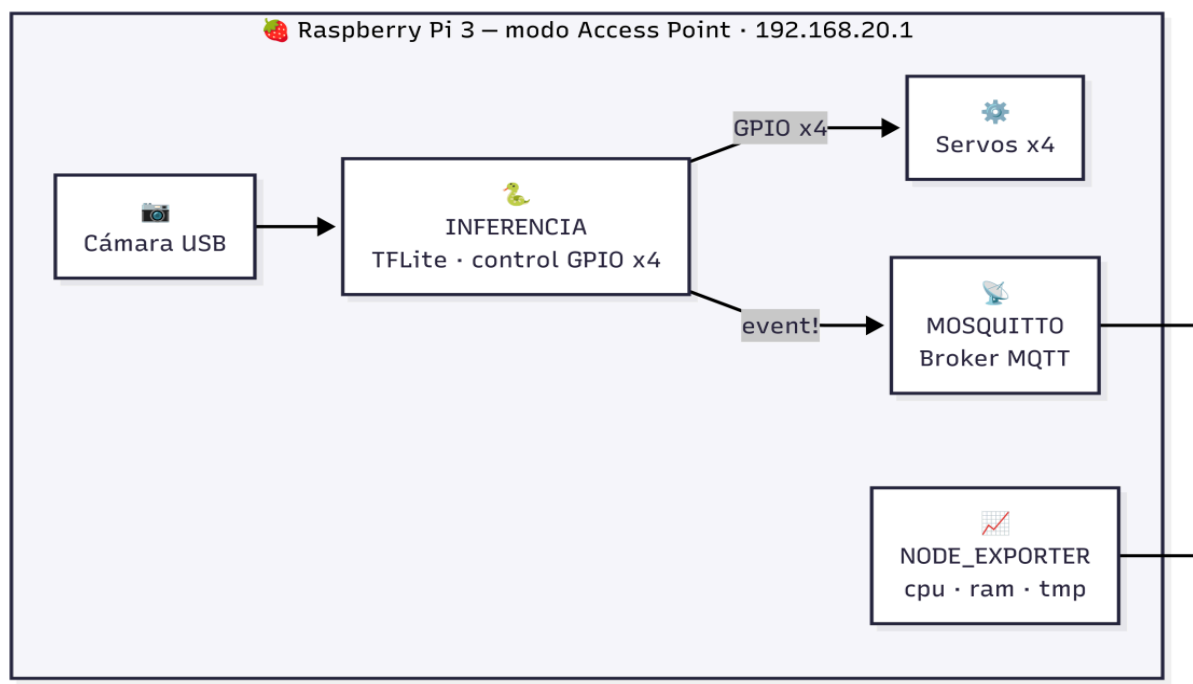
8.1 Software en la Raspberry Pi 3

- **Captura de imagen:** lectura del frame desde la cámara (USB vía OpenCV) al detectar que se depositó un residuo.
- **Inferencia:** modelo de clasificación — obtenido por fine-tuning sobre un modelo pre entrenado y exportado a Tensor Flow Lite en formato int8 — corriendo con tflite-runtime sobre las 4 clases (plástico, papel, vidrio,

orgánico). Por debajo de un umbral de confianza definido, el sistema deriva el residuo a orgánico.

- **Control de actuadores:** script que traduce la clase detectada (o la condición de baja confianza) en la señal PWM correspondiente, usando pigpio — no RPi.GPIO por software, que introduce jitter — para accionar el servo SG90 de la compuerta que corresponde entre las 4 disponibles.
- **Publicación de eventos:** cada clasificación se publica como mensaje MQTT (tópico ecosort/<dispositivo _id>/eventos) al broker Mosquitto, que corre localmente en la propia Pi. Esta publicación se ejecuta en un hilo o proceso separado del loop de inferencia, para no introducir latencia en la ruta caliente.
- **Observabilidad:** node_exporter expone por HTTP las métricas de CPU, RAM y temperatura de la Pi, sin código propio de recolección.

Los procesos de inferencia, Mosquitto y node exporter corren como servicios systemd con reinicio automático (Restart=on-failure), para que una caída puntual no deje el sistema fuera de servicio hasta que alguien lo note.



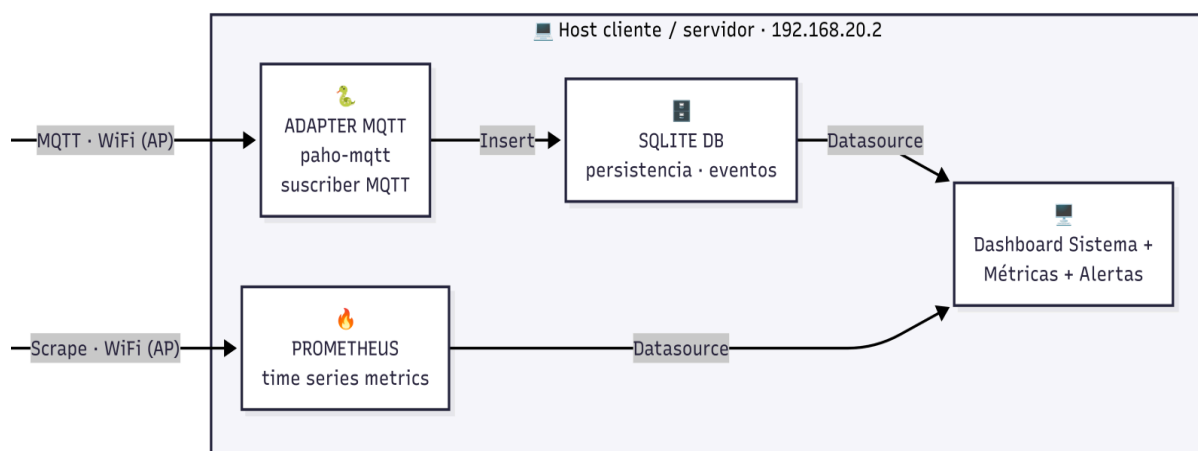
8.2 Software en el dispositivo externo

- **Adaptador MQTT → SQLite:** script en Python (paho-mqtt) suscripto al tópico de eventos, encargado del único INSERT real sobre la base. Sin este proceso no queda historial persistente, porque MQTT no almacena mensajes por diseño.

- **Persistencia:** SQLite (eventos.db, modo WAL) con un esquema mínimo (id, dispositivo id, timestamp, clase, confianza, accionado).
- **Prometheus:** scrapea periódicamente a node_exporter en la Pi a través de la misma red local.
- **Dashboard propio:** detallado en 8.3.

8.3 Sistema Web — Dashboard

- Se sirva en la red local que arma la propia Pi (sin exponerse a internet), accesible desde cualquier dispositivo conectado a esa red.
- Muestre el historial de eventos de clasificación y estadísticas agregadas por clase (cantidad clasificada, distribución, confianza promedio), leyendo directo de eventos.db.
- Muestre el estado de salud del dispositivo (CPU, RAM, temperatura, disponibilidad) consultando la API HTTP de Prometheus, sin tener que scrapear nada por su cuenta.
- No requiere autenticación — se considera aceptable para un prototipo académico dentro de una LAN cerrada, sin exposición externa.
- **Objetivo secundario, si el cronograma lo permite:** sumar herramientas de observabilidad al dashboard, generando un sistema de alerting activo (activado por falta de datos nuevos, dispositivo caído, temperatura sostenida alta).





9. Proximos Avances:

Avance de Octubre

Objetivo: un pipeline funcional end to end, enfocándonos en el módulo INFERENCIA (procesamiento de imágenes y su clasificación) y su comunicación con el host externo y las piezas de hardware.

- Modelo baseline (aún sin ajuste fino de precisión) exportado a TensorFlow Lite, corriendo en la Raspberry Pi 3 y publicando eventos de clasificación por MQTT.
- Tener 4 leds respondiendo a un evento de clasificación, con datos reales o simulados (cada led representando cada tipo de clasificación: plástico / papel / vidrio/ orgánico)
- Comunicación MQTT enviando información al tópico correspondiente.
- Esquema de datos evento (eventos.sql) acordado y congelado entre los tres integrantes.
- Adaptador MQTT → SQLite funcionando, con el dashboard propio leyendo esa base y mostrando los eventos en una tabla y un gráfico simple.
- Circuito de alimentación diseñado y documentado.
- Repositorio con estructura documentada, integración continua necesaria, y tablero de tareas con lo planificado entre los integrantes del equipo.

Avance de Noviembre

Objetivo: sistema integrado y confiable, con validación medible (métricas).

- Precisión de clasificación validada empíricamente, con matriz de confusión y metodología de medición documentada (tamaño del set de prueba por clase, condiciones de iluminación y distancia).
- Manejo de baja confianza implementado y documentado, incluyendo el trade-off de contaminación cruzada que introduce sobre el contenedor de orgánico.
- Las cuatro compuertas funcionan (4 servos implementados) sobre la estructura física terminada (o una versión preliminar que permita mostrar la funcionalidad del proyecto end to end).
- Dashboard propio con historial y estadísticas por clase, y salud del dispositivo visible en tiempo real;
- Documentación completa y proceso de replicación documentado.



10.- Video presentación del Proyecto

Se adjunta el siguiente link para acceder al video presentación de EcoSort:

 [Ing En Computación - Presentación Proyecto EcoSort - TDP2 GrupoG5 \(2026\)](#)